

Document d'Estructures de Dades

Projecte: Extractor de Prototipus de Comportament / Perfils

Assignatura: PROP (Projecte de Programació)

Curs: 2024-2025

Grup: ClusterXX

Autors: - Dari Bachiri, Mohamed - Fiori Porta, Marc - Garvin Cardona, Joan - Gomez Arias,
Jairo - Kumar Aswani, Aman

Entrega 1 — JDK 21 · JUnit 4

Document d'Estructures de Dades	1
Introducció	3
1. Arquitectura i paquets	3
2.Estructures de Dades (Capa de Domini)	4
2.1. Classe Enquesta	4
2.2. Classe Pregunta	6
2.3. Classe Usuari	8
2.4. Classe Perfil	10
2.5. Classes Resposta i Opcio	12
3. Decisions de Disseny i Justificació	13
3.1. Vectors de respostes com a String[]	13
3.2. Persistència en memòria (CtrlPersistencia)	13
3.3. Patró Singleton per a CtrlPersistencia i sessió d'Usuari	13
4. Excepcions i Validació	14
4.1. Excepcions pròpies (Checked)	14
4.2. Centralització de la validació	14
4.3. Validació de respostes	15
5. Resum de Complexitats	16
Taula Comparativa d'Estructures Principals	16
Conclusions	16
Decisions clau:	16
Escalabilitat i evolució:	17
Rendiment:	17

Introducció

Aquest document descriu de manera exhaustiva les estructures de dades implementades al projecte. Per a cada estructura s'indica: l'estructura triada, la justificació, alternatives considerades i per què s'han descartat, i la complexitat asimptòtica de les operacions d'ús habitual.

1. Arquitectura i paquets

El codi segueix una arquitectura en 3 capes amb empaquetatge per domini i controladors:

Capa de Domini (`edu.upc.prop.clusterxx.domini.classes`):

Classes del model (Enquesta, Pregunta, Resposta, Opcio, Usuari, Perfil) i el nucli d'algorismes (DistanceCalculator, KMeans, KMeansPlusPlus, KMedoids, Kluster, ClusterEvaluator, FeatureSpecFactory).

Controladors (`edu.upc.prop.clusterxx.domini.controladors`): CtrlDomini, CtrlAnalisi, CtrlPersistencia, CtrlEnquesta, CtrlPregunta, CtrlResposta, CtrlUsuari, CtrlPerfil.

Capa de Presentació: no està inclosa en aquesta entrega, però CtrlDomini fa de façana per a casos d'ús. La persistència a la primera entrega és en memòria (CtrlPersistencia) mitjançant col·leccions de Java.

2.Estructures de Dades (Capa de Domini)

Aquesta secció descriu l'implementació de les classes d'entitat del paquet `edu.upc.prop.clusterxx.domini.classes`, justificant les estructures de dades triades i la seva complexitat.

2.1. Classe Enquesta

Representa una enquesta amb metadades, un conjunt de preguntes i un registre de participants.

Camps privats

Atribut	Tipus	Descripció
id	String	Identificador únic de l'enquesta
titol	String	Títol de l'enquesta
descripcio	String	Descripció del propòsit de l'enquesta
idCreador	String	Username de l'usuari creador
preguntes	HashMap<String, Pregunta>	Col·lecció de preguntes indexades per ID
participants	ArrayList	Llista de noms d'usuari que han respost l'enquesta

Justificació i ús

Estructures triades: - HashMap<String, Pregunta> per a les preguntes #### Justificació i ús

Preguntes: S'utilitza HashMap<String, Pregunta> per permetre l'accés directe per ID ($O(1)$). Això és essencial per recuperar preguntes ràpidament durant l'edició o visualització sense haver d'iterar tota la llista.

Participants: S'ha triat ArrayList<String> per mantenir el registre d'usuaris. Tot i que la cerca és $O(n)$, per als volums de dades d'aquesta entrega es prioritza la simplicitat i la capacitat de mantenir l'ordre d'inserció cronològic si fos necessari.

Alternatives considerades / descartades

- **LinkedHashMap:** Es va considerar per a preguntes per preservar l'ordre d'inserció visual, però es va descartar en favor del HashMap estàndard, delegant l'ordenació a la capa de presentació o a una llista auxiliar d'IDs.
- **HashSet (participants):** Un HashSet oferiria $O(1)$ per comprovar si un usuari ja ha respost. S'ha descartat temporalment per simplicitat, ja que la llista no s'espera que sigui massiva en aquest prototip.

- **TreeMap:** Descartada perquè no necessitem ordre alfabètic i té més overhead.

Complexitat d'operacions típiques

Operació	Complexitat (pitjor cas)	Justificació
Obtenir pregunta per ID (getPregunta)	$O(1)$	Accés directe en HashMap
Afegir/eliminar pregunta	$O(1)$ amortitzat	Inserció/eliminació en HashMap
Comprovar si usuari ha respost (contains)	$O(n)$	Cerca lineal en ArrayList de n participants
Registrar participació (add)	$O(1)$ amortitzat	Afegir al final de l'ArrayList
Obtenir totes les preguntes (getPreguntes)	$O(p)$	Copiar p preguntes del HashMap a ArrayList

2.2. Classe Pregunta

Modela una pregunta individual, gestionant el seu tipus, opcions (si és qualitativa), rangs i estadístiques (si és numèrica) i les respostes dels usuaris.

Camps privats

Atribut	Tipus	Descripció
id	String	Identificador únic
text	String	Enunciat de la pregunta
tipus	TipusPregunta	Enumeració (NUMERICA, QUALITATIVA_ORDENADA, etc.)
opcions	ArrayList	Llista d'opcions disponibles (per a qualitatives)
maxSeleccions	int	Límit per a multiresposta
valorMinim / valorMaxim	Double	Rang de valors vàlids (per a numèriques)
valorMesAlt / valorMesBaix	Double	Valors extrems registrats (per a normalització)
respostes	HashMap<String, Resposta>	Mapa de respostes indexat per username

Justificació i ús

Opcions: S'utilitza `ArrayList<Opcio>` perquè l'ordre de les opcions és rellevant, especialment en preguntes qualitatives ordenades (escales Likert, etc.). La iteració seqüencial és l'ús més freqüent (mostrar enquesta).

Respostes: S'utilitza `HashMap<String, Resposta>` per accedir ràpidament a la resposta d'un usuari específic ($O(1)$), facilitant la consulta de "la meva resposta".

Camps numèrics: S'afegeixen `valorMesAlt` i `valorMesBaix` per optimitzar el càlcul de distàncies, evitant haver de recórrer totes les respostes per trobar els extrems cada cop que s'executa l'algorisme.

Alternatives considerades / descartades

- **Map<Integer, Opcio>:** Es va descartar mapar les opcions per ID perquè la iteració seqüencial és l'ús més freqüent.
- **ArrayList:** Descartada perquè la cerca per usuari seria $O(n)$.
- **TreeMap<String, Resposta>:** Descartada perquè no necessitem ordre alfabètic d'usuaris.
- **HashSet:** Descartada perquè l'ordre de les opcions és rellevant.

Complexitat d'operacions típiques

Operació	Complexitat (pitjor cas)	Justificació
Validar resposta	$O(n)$	n = opcions o longitud del text
Afegir resposta (inclou act. min/max)	$O(1)$	Inserció en HashMap
Eliminar resposta (inclou recàlcul min/max)	$O(m)$	m = total respostes (recalcular extrems)
Afegir/eliminar opció	$O(o)$	Cerca lineal i eliminació (o opcions)
Obtenir opció per ID	$O(o)$	Cerca lineal entre o opcions
Obtenir resposta per username	$O(1)$	Accés directe en HashMap

2.3. Classe Usuari

Gestiona la informació de l'usuari, les seves creacions, participacions, resultats d'anàlisi (perfils) i la sessió actual.

Camps privats

Atribut	Tipus	Descripció
usuariActual	static Usuari	Referència a l'usuari loguejat (Singleton)
username	String	Identificador únic
password	String	Credencial d'accés
enquestesCreades	List	Llista d'enquestes creades per l'usuari
respostesUsuari	HashMap<String, Resposta>	Respostes donades (indexades per ID Resposta)
perfils	HashMap<String, Perfil>	Perfils assignats (indexats per ID Enquesta)

Justificació i ús

Perfils: S'utilitza `HashMap<String, Perfil>` on la clau és `idEnquesta`. Això permet comprovar en $O(1)$ si l'usuari ja té un perfil assignat per a una enquesta concreta sense iterar una llista.

Respostes: `HashMap<String, Resposta>` permet accés directe per ID de resposta (format: "idPregunta_username").

Sessió: L'ús de `static Usuari usuariActual` simplifica la gestió de sessió per a aquesta entrega, evitant passar l'usuari per tots els controladors. És una solució pragmàtica per al prototip.

Enquestes creades: `List<Enquesta>` manté l'ordre cronològic de creació.

Alternatives considerades / descartades

- **Una segona estructura `Map<idEnquesta, Enquesta>`** per a `enquestesCreades` acceleraria consultes per ID, però `List` és suficient donat que un usuari no sol crear milers d'enquestes.
- **`ArrayList` per respostes/perfils:** Descartada perquè la cerca seria $O(n)$.

Complexitat d'operacions típiques

Operació	Complexitat (pitjor cas)	Justificació
login / logout	$O(1)$	Assignació de variable estàtica
Afegir/obtenir resposta	$O(1)$	Inserció/accés en <code>HashMap</code>

Assignar/obtenir perfil	$O(1)$	Inserció/accés en HashMap
Afegir enquesta creada	$O(1)$	Afegir al final de la List
Eliminar enquesta creada	$O(e)$	Cerca i eliminació en List de e enquestes

2.4. Classe Perfil

Classe (DTO) que emmagatzema el resultat de l'algorisme de clustering per a un usuari en una enquesta específica.

Camps privats

Atribut	Tipus	Descripció
id	Integer	Identificador únic
descripcion	String	Descripció textual
idEnquesta	String	Referència a l'enquesta
clusterIndex	Integer	Índex del clúster assignat
vectorCharacteristic	String[]	Centroide del clúster assignat
clusterSilhouette	Double	Mètrica de qualitat (Silhouette)
clusterMida	Integer	Mida del clúster
nomsPreguntes	List	Metadades per interpretar el vector
algoritme	String	Nom de l'algorisme utilitzat

Justificació i ús

Aquesta classe actua com a **resum immutable** dels resultats de l'anàlisi. Guarda el `vectorCharacteristic` com a `String[]` per consistència amb el controlador d'anàlisi.

Inclou `clusterSilhouette` per mostrar a l'usuari la fiabilitat (pobre, bona, excel·lent) del seu perfil assignat, proporcionant feedback sobre la qualitat del clustering.

La llista `nomsPreguntes` permet interpretar cada dimensió del vector característic, fent el perfil llegible per a l'usuari final.

Alternatives considerades / descartades

- **Array[] fix:** Descartada perquè el nombre de preguntes és variable.
- **HashMap<Integer, String> per noms:** Descartada perquè afegeix complexitat innecessària; l'índex de la llista ja mapeja amb la dimensió del vector.

Complexitat d'operacions típiques

Operació	Complexitat (pitjor cas)	Justificació
Constructor	$O(q)$	Copiar q noms de preguntes
Lectura de dades (<code>getPerfilLlegible</code>)	$O(d)$	Iterar sobre d dimensions

Accès a atributs simples (getters)	$O(1)$	Accès directe a camps
---------------------------------------	--------	-----------------------

2.5. Classes Resposta i Opcio

Classe Resposta

Entitat simple que vincula un valor de resposta amb un usuari i una pregunta.

Camps privats:

Atribut	Tipus	Descripció
id	String	Identificador únic
idPregunta	String	Referència a la pregunta
textResposta	String	Valor de la resposta (serialitzat com String)
usernameUsuari	String	Identificador de l'usuari que respon

Justificació: Es prioritza la simplicitat i serialitzabilitat, emmagatzemant el valor sempre com a String. La interpretació del tipus (convertir a Double o cercar Opcio) es delega a la lògica de Pregunta o DistanceCalculator.

Complexitat: Totes les operacions són $O(1)$ (accés a camps primitius).

Classe Opcio

Representa una opció seleccionable en preguntes qualitatives.

Camps privats:

Atribut	Tipus	Descripció
id	int	Identificador numèric
text	String	Text visualitzat
ordre	Integer	Valor numèric per a l'ordre (nullable)

Justificació: L'atribut ordre permet transformar variables qualitatives ordenades en valors numèrics per al càlcul de distàncies. Si és null, es tracta com a qualitativa nominal.

Complexitat: Totes les operacions són $O(1)$ (accés a camps primitius).

3. Decisions de Disseny i Justificació

Aquest apartat recull les eleccions arquitectòniques i d'implementació més rellevants preses durant el desenvolupament, així com els motius pels quals s'han preferit sobre altres alternatives.

3.1. Vectors de respostes com a String[]

S'ha optat per representar les dades dels usuaris com a **vectors de cadenes de text** (String[]). Aquesta decisió unifica el tractament de dades heterogènies (numèriques, categòriques, text lliure) i delega la interpretació semàntica a la classe FeatureSpec.

Avantatges: - Unificació del tractament de dades heterogènies - Simplicitat en l'implementació dels algorismes de clustering - Facilita la serialització i persistència

Alternativa descartada: Separar les dades per tipus natiu (double[] per a numèriques, Enum per a categòriques, etc.). Tot i que milloraria lleugerament el rendiment i la seguretat de tipus, incrementaria dràsticament la complexitat del codi dels algorismes (necessitat de múltiples matrius de distàncies) i provocaria duplicació de lògica.

3.2. Persistència en memòria (CtrlPersistencia)

Per a aquesta primera entrega, s'ha implementat una **persistència volàtil basada en estructures de dades en memòria** (HashMap, ArrayList).

Escalabilitat: El disseny encapsula l'accés a dades darrere de la classe CtrlPersistencia seguint el patró **Singleton**. Això permetrà, en futures iteracions, substituir l'emmagatzematge en memòria per un sistema de fitxers o una Base de Dades **sense haver de modificar ni una sola línia de codi de la Capa de Domini**.

Avantatges: - Simplicitat per al prototip - Rapidesa de desenvolupament - Facilita el testing unitari - Preparació per a migració futura

3.3. Patró Singleton per a CtrlPersistencia i sessió d'Usuari

S'utilitza el patró Singleton tant per a CtrlPersistencia com per a la gestió de sessió (Usuari.usuariActual).

Justificació: - Garanteix una única font de veritat per a les dades - Simplifica l'accés des de qualsevol controlador - Evita passar referències per tots els mètodes - Adequat per a un prototip amb usuari únic concurrent

Nota: En un sistema multi-usuari real, caldria refactoritzar cap a un sistema de sessions més robust.

4. Excepcions i Validació

La gestió d'errors s'ha dissenyat per garantir la integritat de les dades i proporcionar retroalimentació clara a l'usuari i als desenvolupadors.

4.1. Excepcions pròpies (Checked)

La capa de domini defineix una **jerarquia d'excepcions verificades** (checked exceptions) específiques per a cada cas d'error de negoci. Això obliga a la capa superior a gestionar explícitament els errors previsibles.

Excepcions implementades:

Excepció	Situació
ParametreInvalidException	Paràmetres invàlids (k negatiu, dades buides, etc.)
EnquestaJaExisteixException	Intent de crear enquesta amb ID duplicat
EnquestaNoExisteixException	Intent d'accedir a enquesta inexistent
UsuariNoAutenticatException	Operació que requereix autenticació
CredencialsIncorrectesException	Login amb password incorrecte
EnquestaJaContestadaException	Intent de contestar enquesta ja resposta
RespostaInvalidaException	Resposta que no compleix validació de Pregunta
PermisDenegatException	Intent d'operació sense permisos

4.2. Centralització de la validació

Els casos d'ús es validen principalment al **controlador de domini** (CtrlDomini). Aquesta separació permet:

- Mantenir les classes d'entitat (Usuari, Enquesta) lleugeres i centrades en la seva estructura
- La lògica de negoci i validació complexa resideix als controladors
- Facilita el **testing unitari aïllat** de cada component

Exemple de flux de validació: 1. Petició arriba a CtrlDomini 2. Validació de paràmetres i permisos 3. Si error → Llançar excepció específica 4. Si OK → Delegar a controlador específic (CtrlEnquesta, CtrlResposta, etc.) 5. Retornar resultat o propagar excepció

4.3. Validació de respostes

Cada Pregunta implementa el mètode validarResposta(String) que comprova:

- **NUMERICA:** Valor dins del rang [valorMinim, valorMaxim]
- **QUALITATIVA_SIMPLE/ORDENADA:** Valor coincideix amb text d'una opció
- **QUALITATIVA_MULTIPLE:** Nombre de seleccions $\leq$ maxSeleccions i totes les seleccions són opcions vàlides
- **TEXT_LLIURE:** Text no nul ni buit

Això garanteix la **integritat de les dades** abans d'emmagatzemar-les.

5. Resum de Complexitats

Taula Comparativa d'Estructures Principals

Classe	Estructura	Operació Principal	Complexitat	Alternativa Descartada
Enquesta	HashMap<String, Pregunta>	getPregunta	O(1)	ArrayList (cerca O(n))
Pregunta	HashMap<String, Resposta>	getResposta	O(1)	ArrayList (cerca O(n))
Usuari	HashMap<String, Perfil>	getPerfil	O(1)	ArrayList (cerca O(n))
CtrlPersistencia	HashMap<String, Usuari>	getUsuari	O(1)	TreeMap (overhead)

Conclusions

Aquest document ha presentat de manera exhaustiva les estructures de dades implementades al projecte, justificant cada elecció amb criteris tècnics i analitzant alternatives descartades.

Decisions clau:

1. **HashMap com a estructura dominant** per a entitats amb clau única (usuaris, respostes, perfils, preguntes) gràcies al seu accés O(1), prioritzant el rendiment en les operacions més freqüents.
2. **ArrayList quan l'ordre és semànticament rellevant** (membres de cluster, opcions de pregunta, participants d'enquesta, noms de preguntes en perfils), acceptant complexitat O(n) en cerques ocasionals a canvi de simplicitat i manteniment de l'ordre d'inserció.
3. **Patró Singleton** aplicat a CtrlPersistencia i sessió d'usuari per centralitzar l'accés a dades i simplificar la gestió en el prototip, amb consciència de les limitacions per a un sistema multi-usuari real.
4. **Còpies defensives sistemàtiques** per protegir la integritat de les dades, acceptant cost O(n) com a inversió en robustesa del sistema.
5. **Vectors de respostes com String[]** per unificar el tractament de dades heterogènies, delegant la interpretació semàntica a les especificacions de característiques.
6. **Jerarquia d'excepcions verificades** per forçar la gestió explícita d'errors i millorar la robustesa del sistema.
7. **Encapsulació de persistència** darrere de CtrlPersistencia per facilitar la migració futura a sistemes de fitxers o bases de dades sense impacte en la capa de domini.

Escalabilitat i evolució:

El disseny implementat té en compte l'evolució futura del sistema: - La persistència en memòria és fàcilment substituïble - Les estructures són escalables per a volums moderats de dades - La separació de capes facilita l'extensió amb noves funcionalitats